

Optimal spanning tree reconstruction in symbolic regression

IA pour la Science

April 16, 2026

Table 1: Probabilities in the adjacency matrix generate the directed graph

arity	prim.	*	+	ln	sin	$\times$	exp	x
1	*	0.2	0.7	0.5	0.4	0.5	0.3	0.2
3	+	0.3	0.2	1.0	0.8	0.6	0.3	0.7
1	ln	0.3	0.2	0.0	0.0	0.1	0.5	0.5
1	sin	0.1	0.4	0.0	0.5	0.9	0.2	0.5
2	$\times$	0.3	0.0	0.3	0.5	0.0	0.8	0.6
1	exp	0.3	0.3	0.4	0.1	0.5	0.4	0.4

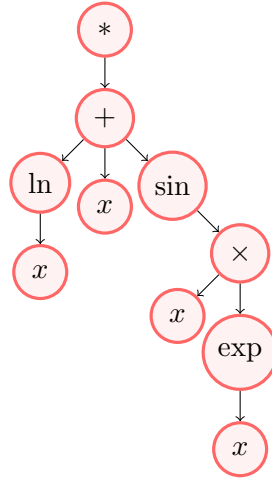


Figure 1: The regression model structure is a directed graph

The proposed solution is based on the relaxed version of k -MST problem, which transforms to the PCST problem with constant prizes, the same for all vertices. The fast algorithm for PSCT is described in [1]. An alternative solution is based on $(2 - \varepsilon)$ approximation algorithm for PSCT problem. It is compared with the other algorithms, including the tree depth-first traverse, tree breadth-first traverse, Prim's algorithms.

1 The regression model selection problem

One has to select a regression model φ from a set of alternative models. The model fits a sample set $D = \{(x_i, y_i)\}$ and minimises the error

$$\hat{\varphi}(D) = \arg \min_{\varphi} \sum_{i=1}^m (\varphi(x_i) - y_i)^2. \quad (1)$$

The model is a superposition of the primitive functions from some given set. Fig. 1 shows an example. A structure of the model φ , a superposition corresponds to some graph $G = (V, E)$ where the primitive functions are placed in the vertices V . The root vertex is denoted by $*$. The model is $\varphi(D) = \ln(x) + x + \sin(x \times \exp(x))$. Its structure as the graph adjacency matrix is provided in Table 1. The primitive functions are listed in the first row. The elements of the matrix are probabilities of the edges E of the tree. The bold typesetting highlights the edges of the reconstructed tree M , which form the superposition φ . To reconstruct the model structure φ as superposition that is defined by the tree M one needs only the graph representation G and the primitive functions.

State the problem of the model structure reconstruction. There given a collection of sample sets $\{D_k\}$. Each sample set D_k corresponds to its model to fit. This model has the structure M_k . So there is a collection of pairs: a sample set and its model structure, $\{(D_k, M_k)\}$. Denote by P a map, which predicts the probabilities of nodes in the graph G using a sample set D . To define a model $\varphi(D)$ one has to reconstruct the model structure M from the graph G . Denote by R this tree reconstruction algorithm. The regression model $\hat{\varphi}(D)$, which solves the problem (1), is defined by $\hat{M} = R(P(D))$. Since the tree M plays the main role, this paper sets the quality criterion of the tree reconstruction algorithm as follows:

$$\min_{M_k \in G} \frac{1}{K} \sum_{k=1}^K [\hat{M}_k = M_k].$$

The reconstructed tree must be equal to the given tree to guarantee the selected regression model fits its sample set.

2 The superposition tree reconstruction problem

The key problem of this work is to propose and analyse the tree reconstruction algorithms. Each reconstructed tree defines a superposition from the previous section. There given the directed weighted graph $G = (V, E)$ with the colored vertices v_i and the root vertex r . Every vertex $v_i \in V$ has its color $t(v_i) = t_i$. Every edge $e_i \in E$ has its weight $w(e_i) = c_i \in [0, 1]$.

The goal is to reconstruct a minimum-weight directed tree with the root r . It must cover at least k vertices in the given graph G . And number of tail edges, out-coming from a vertex v_i of the tree must be less than or equal to t_i . The root r has one edge, $t_r = 1$.

Formulate this statement in the form of the linear programming problem with integer constraints:

$$\begin{aligned}
& \underset{\substack{x_e, z_S \\ e \in E, S \subseteq V \setminus \{r\}}}{\text{minimize}} && \sum_{e \in E} c_e x_e \\
& \text{s.t.} && \sum_{\substack{e \in \delta(S): \\ e = (*, v_i), v_i \in \delta(S)}} x_e + \sum_{T: T \supseteq S} z_T \geq 1, && S \subseteq V \setminus \{r\}, \\
& && \sum_{e \in E: e = (*, v)} x_e \leq 1, && v \in V, \\
& && \sum_{e \in E: e = (v, *)} x_e \leq t_i, && v \in V, \\
& && \sum_{S \subseteq V \setminus \{r\}} |S| z_S \leq n - k, \\
& && x_e \in \{0, 1\}, && e \in E, \\
& && z_S \in \{0, 1\}, && S \subseteq V \setminus \{r\},
\end{aligned} \tag{2}$$

where $x_e = 1$ if edge e is included into the final superposition and $x_e = 0$ otherwise, $z_S = 1$ for all vertices excluded from the final superposition. Denote by $e = (*, v)$ a directed edge with the tail v . Denote by $e = (v, *)$ a directed edge with the head v .

Every constrain in (2) has its specific purpose. The first constraint defines the structure of the solution graph as a tree with the root r . The second constraint defines the orientation of the tree: every vertex has no more than one incoming edge. The third constraint defines the arity of the used primitive functions, so the number of edges that have a certain vertex as their source is fixed. The fourth constraint states that the final tree has at least k vertices. If all weights are non-negative, the fourth constraint on the minimal number of vertices takes a more strict form: the number of vertices should be exactly k . However, the weaker constraint allows to find possible connections with other optimization problems. The exact form of the constraints in (2) has the same goal.

3 The k -MST and PCST algorithms for tree reconstruction

Definition 1 *k -minimum spanning tree (k -MST).* There given a weighted graph $G = (V, E)$ with root r and edge weights $w(e_i) = c_i \geq 0$, $e_i \in E$. Construct a minimum-weight directed tree with root r , which covers at least k vertices in G .

If the same problem is formulated for the directed graphs, the final tree with root r should be directed. The linear programming problem for the directed k -MST excludes the third condition in (2). In such form the k -MST problem is different from the original superposition tree reconstruction problem (2) by the absence of the third constraint on the arity of

primitive functions. It is equivalent to the constraint on number of edges out-coming from a vertex.

Definition 2 *Prize-collecting Steiner tree (PCST).* There given a weighted graph $G = (V, E)$ with root r and edge weights $w(e_i) = c_i \geq 0$, $e_i \in E$, where every vertex $v_i \in V$ is assigned with a prize $\pi(v_i) = \pi_i \geq 0$. Construct a tree T with root r , which minimizes the functional:

$$\sum_{e \in E} c_e x_e + \sum_{S \subseteq V \setminus \{r\}} \pi(S) z_S,$$

where $x_e \in \{0, 1\}$, $x_e = 1$ if $e \in E$ is included in the tree T , $z_S \in \{0, 1\}$, $z_S = 1$ for all vertices excluded from tree T $S = V \setminus V(T)$ and $\pi(S) = \sum_{v \in S} \pi(v)$.

In case of directed graphs this problem generalizes to the asymmetric A-PCST problem. The linear programming problem for A-PCST takes the form:

$$\begin{aligned} & \underset{x_e, z_S}{\text{minimize}} && \sum_{e \in E} c_e x_e + \sum_{S \subseteq V \setminus \{r\}} \pi(S) z_S \\ & \text{s.t.} && \sum_{\substack{e \in \delta(S): \\ e = (*, v_i), v_i \in \delta(S)}} x_e + \sum_{T: T \supseteq S} z_T \geq 1, && S \subseteq V \setminus \{r\}, \\ & && \sum_{e \in E: e = (*, v)} x_e \leq 1, && v \in V, \\ & && x_e \in \{0, 1\}, && e \in E, \\ & && z_S \in \{0, 1\}, && S \subseteq V \setminus \{r\}. \end{aligned} \quad (3)$$

If the last constraint from (2) is included into the optimized functional, the k -MST and A-PCST problems have equivalent constraints and only differ in the optimized functional. Such transformation is possible according to the Karush-Kuhn-Tucker conditions and [2]. If the prize values are equivalent $\pi(v) = \lambda$ the only difference is the constant term $\lambda(n - k)$. So the optimization problems k -MST and A-PCST take the forms:

$$\begin{aligned} & \underset{x_e, z_S}{\text{minimize}} && \sum_{e \in E} c_e x_e + \lambda \left(\sum_{S \subseteq V \setminus \{r\}} |S| z_S - (n - k) \right), \\ & \underset{x_e, z_S}{\text{minimize}} && \sum_{e \in E} c_e x_e + \lambda \sum_{S \subseteq V \setminus \{r\}} |S| z_S. \end{aligned}$$

The constant λ stands for non-negative Lagrange multiplier in the k -MST problem and for vertex prize in the A-PCST. There are several algorithms to solve the PCST problem, but not A-PCST. A possible workaround is to release the constraints on the graph orientation so the PCST algorithm could reconstruct the tree orientation later.

4 The $(2 - \varepsilon)$ -approximation algorithm for constrained forest problem

An overview of techniques for constrained forest problems is provided in [3]. This research selects relevant results. There given a weighted undirected graph $G = (V, E)$. All its weights $w(e_i) = c_i \geq 0$, $e_i \in E$. There given some function $f : 2^V \rightarrow \{0, 1\}$. State the linear programming problem with integer constraints:

$$\begin{aligned} & \underset{x_e: e \in E}{\text{minimize}} && \sum_{e \in E} c_e x_e \\ & \text{s.t.} && x(\delta(S)) \geq f(S), && S \subset V, S \neq \emptyset, \\ & && x_e \in \{0, 1\}, && e \in E, \end{aligned} \tag{4}$$

where $x(\delta(S)) = \sum_{e \in \delta(S)} x_e$; $x_e = 1$ if edge e is included into the final set. The function $\delta(S)$ stands for all edges from E such that only one of the connected vertices is included in S .

Assume the map f , which satisfies

$$f(V) = 0, \underbrace{f(S) = f(V \setminus S)}_{\text{symmetry}}, \underbrace{A, B \subset V : A \cap B = \emptyset, f(A) = f(B) = 0 \rightarrow f(A \cup B) = 0}_{\text{disjunctivity}}.$$

In these conditions are satisfied, f specifies the number of edges, which starts in the set of vertices S . For example, for the minimum matching problem $f(S) = 1$ if and only if $|S| \bmod 2 = 1$.

Lemma 1 *Let $B \subseteq S \subset V$. Then $f(S) = 0$ and $f(B) = 0$ leads to $f(S \setminus B) = 0$.*

A problem with such description is the *optimal forest search problem with correct constraints*. Such problem statement (4) with appropriate map f fits many well-known weighted graph problems, e.g. minimum backbone search, *st*-path, the Steiner problem on the minimum tree. The last problem is NP-complete, so apply an approximate algorithm.

Definition 3 (*α -approximation algorithm*) *A heuristic polynomial algorithm that delivers a solution for some optimization problem is called α -approximation if it guarantees a constraint-satisfying solution to this optimization problem with a factor less or equal to α , so the solution is different from the optimal one no more than by α times in terms of the optimized functional.*

To propose an appropriate approximate algorithm, the integer constraints in (4) should be relaxed:

$$\begin{aligned} & \underset{x_e: e \in E}{\text{minimize}} && \sum_{e \in E} c_e x_e \\ & \text{s.t.} && \sum_{e \in \delta(S)} x_e \geq f(S), && S \subset V, S \neq \emptyset, \\ & && x_e > 0, && e \in E, \end{aligned} \tag{5}$$

The dual problem takes the form:

$$\begin{aligned}
& \underset{y_S: S \subset V, S \neq \emptyset}{\text{maximize}} && \sum_{S \subset V} f(S) y_S \\
& \text{s.t.} && \sum_{S: e \in \delta(S)} y_S \leq c_e, && e \in E, \\
& && y_S > 0, && S \subset V, S \neq \emptyset,
\end{aligned} \tag{6}$$

regarding a complementary slackness condition: $y_S \cdot \left(\sum_{e \in \delta(S)} x_e - f(S) \right) = 0, \quad S \subset V.$

Denote the set of vertices $A = \{v \in V : f(\{v\}) = 1\}$. Propose an adaptive greedy $\left(2 - \frac{2}{|A|}\right)$ — approximation algorithm for problems of the form (4). The algorithm consists of two stages. On the first stage it greedily combines clusters of vertices increasing the dual variables y_S . Initially every vertex belongs to its own cluster. If the next edge e reaches equality in the constraints in (6), this edge is added to the set S and the connected clusters will be merged. This stage is similar to Kruskal minimum spanning tree algorithm. In the second stage some edges are removed from the final set S . If the edge pruning does not violate the constraints, this edge is to be removed.

The pseudo-code for the described algorithm is provided in the Appendix of this paper. The index Z_{DRLP} in Algorithm 1 stands for dual-relaxed linear programming. The initial value of $F := \emptyset$ in 1 is equivalent to the assumption $x_e = 0 \quad e \in E$. According to the slackness conditions $y_S = 0, \quad S \subset V, S \neq \emptyset$.

At any step of the algorithm, cluster $\mathcal{C}$ contains two components $\mathcal{C} = \mathcal{C}_i \cup \mathcal{C}_a$, where $C \in \mathcal{C}_a$ if $f(C) = 1$ and $C \in \mathcal{C}_i$ otherwise. Let's call $\mathcal{C}_a$ an active component. The variables $d(v)$ in this algorithm are related to the variables y_S from (6) as $d(i) = \sum_{S: i \in S} y_S$.

Analyse two different components $C_q, C_p, C_q \cap C_p = \emptyset$ on some iteration of the first stage of the algorithm. All y_S should be evenly by some ε without violating the constraints

$$\sum_{S: e \in \delta(S)} y_S \leq c_e.$$

In terms of $d(v)$, this condition takes the form $\sum_{S: e \in \delta(S)} y_S = d(v_1) + d(v_2), \quad e = (v_1, v_2),$

so $y_S = 0$ for any S , such that $v_1, v_2 \in S$ because the components only grow on the first stage. Increasing some of the components by ε leads to the equation

$$d(v_1) + d(v_2) + \varepsilon \cdot (f(C_q) + f(C_p)) \leq c_e, \quad e = (v_1, v_2),$$

which leads to the formula used in line 10 of the Algorithm 1. In the case when the next edge is included into the component, the sum $\sum_{S: e \in \delta(S)} y_S$ will not increase, so the constraints are satisfied.

Edges that can be removed from F without addition of new active components are removed on the second stage of the algorithm. The following lemma defines properties of connected components in F' .

Lemma 2 *For every connected component N from F' the equation holds: $f(N) = 0$.*

The following theorem states that the solution derived by the described algorithm is meeting the constraints of the original linear programming problem.

Theorem 1 *The edge set F' derived by Algorithm 1 meets all the constraints of the original problem (4).*

Lemma 3 *Denote graph H where every vertex corresponds to one of the connected components $C \in \mathcal{C}$ on the fixed step of the algorithm. Edge (v_1, v_2) is present if there exists an edge $\hat{e}$ of the original graph included in F' : $\hat{e} \in F'$, so the graph H is a forest. There are no leaf vertices within H such that correspond to inactive vertices in the original graph.*

Theorem 2 *Algorithm 1 is an α -approximate algorithm for problem (4) with $\alpha = 2 - \frac{2}{|A|}$ where $A = \{v \in V : f(\{v\}) = 1\}$.*

Despite this theoretical basis, there is no appropriate function f to state the PCST problem as referenced in (4). To be applicable in these conditions, the Algorithm 1 need several modifications.

5 The upgraded problem statement for PCST

As in the A-PCST case, the relaxed form of the linear programming problem PCST takes form:

$$\begin{aligned}
& \underset{x_e, s_v}{\text{minimize}} && \sum_{e \in E} c_e x_e + \sum_{v \in V \setminus \{r\}} (1 - s_v) \pi_v \\
& \text{s.t.} && \sum_{e \in \delta(S)} x_e \geq s_v, && S \subseteq V \setminus \{r\}, v \in S, \\
& && x_e \geq 0, && e \in E, \\
& && s_v \geq 0, && v \in V \setminus \{r\}.
\end{aligned} \tag{7}$$

This problem statement is different from the original one (3), but it is possible to align the k -MST problem with it. Indicators s_v show that vertex v is included in the tree.

The dual problem takes form:

$$\begin{aligned}
& \underset{y_S}{\text{maximize}} && \sum_{S \in V \setminus \{r\}} y_S \\
& \text{s.t.} && \sum_{S: e \in \delta(S)} y_S \leq c_e, && e \in E, \\
& && \sum_{S \subseteq T} y_S \leq \sum_{v \in T} \pi_v, && T \subset V \setminus \{r\}, \\
& && y_S \geq 0, && S \subset V \setminus \{r\}.
\end{aligned} \tag{8}$$

Algorithm 2 solves this problem. It is similar to Algorithm 1. The dual variables should be updated at an even rate with additional constraints. Then ε will take the minimum of two values, according to the both groups of constraints. The broader analysis of the approximation properties of the upgraded algorithm is provided in [3]. Algorithm 2 is an α -approximate algorithm for PCST problem with $\alpha = 2 - \frac{2}{n-1}$, where n is the number of vertices in the graph G .

6 Computational experiment

The main goal of the experiment is to reconstruct the correct superposition tree. The algorithms used for reconstruction are listed below.

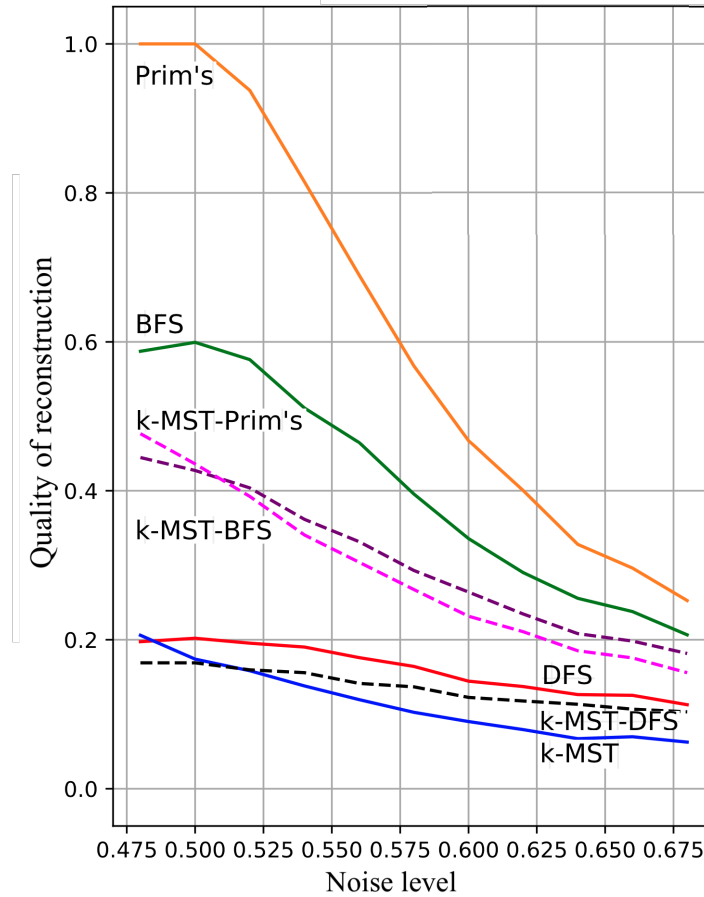


Figure 2: Quality of the reconstruction algorithms with primitive functions of small arities and unordered inputs

DFS, BFS. Greedy tree depth-first traverse and Greedy tree breadth-first traverse algorithms. To traverse the edges with highest weights is equivalent to select the most probable

path. The traverse algorithm stops when the number of tail edges out-coming from some vertex equals the arity of the corresponding function.

Prim’s algorithm. This algorithm reconstructs the minimum spanning tree for a graph with additional constraints on the primitive functions’ arity. These constraints assign the minimum weight edge. After a vertex is added, all tail edges of this vertex are excluded to preserve direction of the tree. If the number of edges starting in some vertex exceeds the corresponding arity, the rest of edges are excluded from the set of possible edges in this vertex. The algorithm is independent of the traverse procedure. In case of small noise in the adjacency matrix, this algorithm is able to reconstruct the superposition tree without errors. Unfortunately, the algorithm fails if a variable is used several times.

Table 2: Quality of reconstruction algorithms with uniform noise near to 0.5

Algorithm, noise	0.50	0.52	0.54	0.56	0.58
DFS	0.20	0.20	0.19	0.18	0.16
BFS	0.60	0.58	0.51	0.46	0.40
Prim’s	1.00	0.94	0.81	0.69	0.57
k -MST	0.17	0.16	0.14	0.12	0.10
k -MST-DFS	0.17	0.16	0.16	0.14	0.14
k -MST-BFS	0.43	0.40	0.36	0.33	0.29
k -MST-Prim’s	0.44	0.39	0.34	0.33	0.27

Algorithms based on PCST. The adjacency matrix M must be transformed to the undirected form. Use the square matrix M' without the last column. PCST takes the adjacency matrix $1 - \frac{1}{2}(M' + M'^T)$ with prize value 0.5 for every vertex. The prize value is equal to 0.5 because with smaller values the tree will be truncated: if the noise is equal to 0.5 some vertices might be pruned by error. In case of greater prize values the PCST tree might include unnecessary vertices. The tree is reconstructed with one of the described algorithms. The results of PCST can be used as prior for other approaches, $M' := \frac{1}{2}(M'_{\text{PCST}} + M')$, so the PCST results update M' .

The data generation procedure has the following assumptions: the arities of the function are generated by the Binomial distribution so there are many functions with small arity, all primitive functions have only one input. Any case with partial reconstruction is treated as an error. The quality of reconstruction algorithms is $\frac{1}{K} \sum_{k=1}^K [R(\bar{N}(M_k)) = M_k]$, where R is the reconstruction algorithm and $\bar{N} = (N - \min(N)) / (\max(N) - \min(N))$ is the normalized noise matrix. The matrix N is generated as $N(M) = M + U(-\alpha, \alpha)$. The random generator returns a matrix with the same shape as M where every element is independent variable from the uniform distribution in the segment $[-\alpha, \alpha]$.

Here is the list of seven compared algorithms: DFS, BFS, Prim’s algorithm, k -MST via PCST, k -MST + DFS, k -MST + BFS, k -MST + Prim’s algorithm. Fig. 2 shows the error of the reconstruction algorithms with the noise close to 0.5 threshold. The best results are

delivered by the Prim’s algorithm. The second best solution is based on BFS. Table 2 accompanies Fig. 2 and shows the reconstruction quality of seven algorithms for the border noise values 0.50–0.58.

References

- [1] Hegde, C., P. Indyk, and L. Schmidt. 2014. A fast, adaptive variant of the Goemans-Williamson scheme for the prize-collecting steiner tree problem. *Workshop of the 11th DIMACS Implementation Challenge* 2:3.
- [2] Ras, C., K. Swanepoel, and D. A. Thomas. 2017. Approximate Euclidean Steiner trees. *Journal of Optimization Theory and Applications* 172(3):845–873.
- [3] Goemans, M. X. and D. P. Williamson. 1995. A general approximation technique for constrained forest problems. *SIAM Journal on Computing*, 24(2):296–317.

Appendix 1: Tree reconstruction algorithms

Algorithm 1: $(2 - \varepsilon)$ -approximation algorithm for problem (4)

Data: Weighted undirected graph $G = (V, E)$ with non-negative weights $c_i \geq 0$;
map f

Result: Forest F' ; optimized in problem functional (4) value Z_{DRLP}

Stage 1, Merging

begin

$F \leftarrow \emptyset$

$Z_{\text{DRLP}} \leftarrow 0$

$\mathcal{C} \leftarrow \{\{v\} : v \in V\}$

foreach $v \in V$ **do**

$d(v) \leftarrow 0$

while $\exists C \in \mathcal{C} : f(C) = 1$ **do**

$e^* = \arg \min_{\substack{e=(i,j): \\ i \in C_p \in \mathcal{C}, j \in C_q \in \mathcal{C}, C_p \neq C_q}} \varepsilon(e) \text{ where } \varepsilon(e) = \frac{c_e - d(i) - d(j)}{f(C_p) + f(C_q)}$

$F \leftarrow F \cup e^*$

foreach $C \in \mathcal{C}$ **do**

foreach $v \in C$ **do**

$d(v) \leftarrow d(v) + \varepsilon(e^*) \cdot f(C)$

$Z_{\text{DRLP}} \leftarrow Z_{\text{DRLP}} + \varepsilon(e^*) \sum_{C \in \mathcal{C}} f(C)$

$\mathcal{C} \leftarrow \mathcal{C} \setminus \{C_p\} \setminus \{C_q\} \cup \{C_p \cup C_q\}$ (e^* connects components C_q and C_p)

Stage 2, pruning

$F' \leftarrow \{e \in F : \exists N \in (V, F \setminus \{e\}), f(N) = 1\}$, where N is the connected component

Algorithm 2: $(2 - \varepsilon)$ -approximate algorithm for PCST problem

Data: Weighted undirected graph $G = (V, E)$ with non-negative edges' weights

$c_i \geq 0$, prizes $\pi_i \geq 0$ and root r

Result: Tree F' including vertex r

Stage 1, Merging

begin

$F \leftarrow \emptyset$

$Z_{\text{DRLP}} \leftarrow 0$

$\mathcal{C} \leftarrow \{\{v\} : v \in V\}$

foreach $v \in V$ **do**

 Remove markup from v

$d(v) \leftarrow 0$

$w(\{v\}) \leftarrow 0$

if $v = r$ **then** $\lambda(\{v\}) \leftarrow 0$

else $\lambda(\{v\}) \leftarrow 1$

while $\exists C \in \mathcal{C} : \lambda(C) = 1$ **do**

$e^* = \arg \min_{\substack{e=(i,j): \\ i \in C_p \in \mathcal{C}, j \in C_q \in \mathcal{C}, C_p \neq C_q}} \varepsilon_1(e) \text{ where } \varepsilon_1(e) = \frac{c_e - d(i) - d(j)}{\lambda(C_p) + \lambda(C_q)}$

$C^* = \arg \min_{C: C \in \mathcal{C}, \lambda(C)=1} \varepsilon_2(C) \text{ where } \varepsilon_2(C) = \sum_{i \in C} \pi_i - w(C)$

$\varepsilon = \min(\varepsilon_1(e^*), \varepsilon_2(C^*))$

foreach $C \in \mathcal{C}$ **do**

$w(C) \leftarrow w(C) + \varepsilon \cdot \lambda(C)$

foreach $v \in C$ **do**

$d(v) \leftarrow d(v) + \varepsilon \cdot \lambda(C)$

if $\varepsilon_1(e^*) > \varepsilon_2(C^*)$ **then**

$\lambda(C^*) \leftarrow 0$ Mark all unmarked vertices from C^* with C^* .

else

$F \leftarrow F \cup e^*$

$\mathcal{C} \leftarrow \mathcal{C} \setminus \{C_p\} \setminus \{C_q\} \cup \{C_p \cup C_q\}$ (e^* connects components C_q and C_p)

$w(C_p \cup C_q) \leftarrow w(C_p) + w(C_q)$

if $r \in C_p \cup C_q$ **then** $\lambda(C_p \cup C_q) \leftarrow 0$

else $\lambda(C_p \cup C_q) \leftarrow 1$

Stage 2, pruning

F' is derived from F by dropping the maximum number of edges meeting the constraints:

1. All unmarked vertices are connected with root r .
 2. If vertex marked with C is connected with root r , all other vertices marked with C should be connected with root r as well.
-

Algorithm 3: Superposition tree reconstruction with Prim's algorithm

Data: Noised superposition matrix $M \in \mathbb{R}_+^{n \times (n+1)}$, list l with $n - 1$ arity values for used functions

Result: Superposition matrix M_{res} with correct arities

begin

$l \leftarrow [1] + l$ (add 1 to the list)

$M' \leftarrow$ zero matrix of shape $n \times (n + 1)$

$used \leftarrow \{0\}$

$edges \leftarrow \emptyset$

foreach $j \in range(0, n)$ **do**

if $j \notin used$ **then**
 $edges \leftarrow edges \cup (0, j, M[0][j])$ (from, to, weight)

while $edges \neq \emptyset$ **do**

 Find tuple $(from, to, w)$ maximizing the edge weight w over all $edges$

foreach $j \in used$ **do**

$M[to][j] = 0$

foreach $j \in range(0, n)$ **do**

if $j \notin used$ **then**
 $edges \leftarrow edges \cup (to, j, M[to][j])$ (from, to, weight)

if $to \neq n$ **then**

$edges \leftarrow edges \setminus (from, to, w)$
 $l[from] \leftarrow l[from] - 1$

 Remove from $edges$ all tuples (i, j, w) with $j = to$

if $l[to] = 0$ **then**

 Remove from $edges$ all tuples (i, j, w) with $i = from$

Algorithm 4: Superposition tree reconstruction with algorithm for the PCST problem

Data: Noised superposition matrix $M \in \mathbb{R}_+^{n \times (n+1)}$, list l with $n - 1$ arity values for used functions

Result: Superposition matrix M_{res} with correct arities

begin

Drop the last column from matrix M to derive matrix M'

$$M'_{new} = 1 - \frac{M' + M'^T}{2}$$

$$M'_{pcst} = PCST(M'_{new}, 0.5)$$

Add zero column to the M'_{pcst} on the right to derive M_{pcst}

Reconstruct the tree from M_{pcst} with some traverse procedure from the root vertex to derive M_{res}

Appendix 2: Proofs to the lemmas and theorems

Proof 1 (to lemma 1) *The Symmetry property leads to $f(V \setminus S) = 0$. Since $V \setminus S \cap B = \emptyset$, the disjunctivity property leads to $f((V \setminus S) \cup B) = 0$. According to the symmetry property, the equation holds:*

$$f(V \setminus ((V \setminus S) \cup B)) = f(S \setminus B) = 0.$$

Proof 2 (to lemma 2) *Recall that F' is constructed from F via pruning. Hence there is a connected component $C \in F$ such that $N \subseteq C$. The algorithm has stopped, so $f(C) = 0$. All the edges $\delta(N)$ which started from N before pruning and were pruned. Then there is no component $\hat{N}$ such that $f(\hat{N}) = 1$ present in $(V, E \setminus \{e\})$, $e \in \delta(N)$. Denote the C components derived via edge pruning from $\delta(N)$ as $N, N_1, \dots, N_{|\delta(N)|}$, then $f(N_i) = 0$. According to disjunctive property $f(\bigcup_{i=1}^{|\delta(S)|} N_i) = 0$. Hence, according to Lemma 1 $f(N) = f(V \setminus \bigcup_{i=1}^{|\delta(S)|} N_i) = 0$.*

Proof 3 (to theorem 1) *Assume there is empty set $S \subset V$, $S \neq \emptyset$, such that $\sum_{e \in \delta(S)} x_e < f(S)$. Then $f(S) = 1$. For every connected component $C_1, \dots, C_m$ from (V, F') one of the following equations stand: $C_i \subseteq S$ or $C_i \cap S = \emptyset$ according to $\sum_{e \in \delta(S)} x_e = 0$. According to Lemma 2 and disjunctive property, $f(S) = f(\bigcup_j C_{i_j}) = 0$, which is contrary to the original assumption $f(S) = 1$.*

Proof 4 (to theorem 2) *Recall the inequality:*

$$Z_{LP}^* \leq \sum_{e \in F'} c_e \leq (2 - \frac{2}{|A|}) Z_{DRLP} \leq (2 - \frac{2}{|A|}) Z_{LP}^*.$$

1. *The first part holds according to the Z_{LP}^* definition and the fact that F' meets the constraints according to Theorem 1.*
2. *The last part holds because for the optimal solution Z_{LP}^* of the problem 4 the following inequality holds due to the below constraints of the dual problem: $Z_{DRLP} = \sum_{S \subset V} y_S \leq Z_{RLP}^* \leq Z_{LP}^*$.*
3. *The middle part should be proven explicitly.*

After the Algorithm 1 stops $c_e = \sum y_S$, so the following equation stands:

$$\sum_{e \in F'} c_e = \sum_{e \in F'} \sum_{S: e \in \delta(S)} y_S = \sum_{S \subset V} y_S \cdot |F' \cap \delta(S)|.$$

So the following inequality should will be proved by induction:

$$\sum_{e \in F'} c_e = \sum_{S \subset V} y_S \cdot |F' \cap \delta(S)| \leq (2 - \frac{2}{|A|}) Z_{DRLP} = (2 - \frac{2}{|A|}) \sum_{S \subset V} y_S.$$

Basis case: On the first step of the algorithm $y_S = 0$.

Inductive step: assume the induction hypothesis that for a particular step k , the inequality holds. On the $(k + 1)$ step the left hand side is increased by $\varepsilon \sum_{S \in \mathcal{C}_a} |F' \cap \delta(S)|$, where $\mathcal{C}_a$ stays for all active components and $f(C) = 1$. The right hand side will be increased by $\varepsilon(2 - \frac{2}{|A|}) \cdot |\mathcal{C}_a|$. Let's focus on the following inequality:

$$\sum_{S \in \mathcal{C}_a} |F' \cap \delta(S)| \leq (2 - \frac{2}{|A|}) \cdot |\mathcal{C}_a|.$$

Denote the number of edges starting in S as $d(S) = |F' \cap \delta(S)|$. So

$$\sum_{S \in \mathcal{C}_a} d(S) = \sum_{S \in \mathcal{C}} d(S) - \sum_{S \in \mathcal{C}_i} d(S) \leq 2(|\mathcal{C}_a| + |\mathcal{C}_i| - 1) - \sum_{S \in \mathcal{C}_i} d(S),$$

where $\mathcal{C} = \mathcal{C}_i \cup \mathcal{C}_a$. The last inequality holds because F' defines a forest in the original graph. The last step is to prove that

$$\sum_{S \in \mathcal{C}_i} d(S) \geq 2|\mathcal{C}_i|, \tag{9}$$

which implies

$$\sum_{S \in \mathcal{C}_a} d(S) \leq 2(|\mathcal{C}_a| + |\mathcal{C}_i| - 1) - 2|\mathcal{C}_i| = 2(1 - \frac{1}{|A|}) \cdot |\mathcal{C}_a| \leq 2(1 - \frac{1}{|A|}) \cdot |\mathcal{C}_a|,$$

which holds because the number of clusters does not increase through time, or equivalently $|A| \geq |\mathcal{C}_a|$. According to the Lemma 3 proves the inequality (9), so the second part of the original statement is also correct.

Proof 5 (to lemma 3) The following lemma is needed to prove inequality (9). Recall that lone inactive vertices are ignored due to the zero power of every inactive vertex, so they all can be subtracted in the inequality $\sum_{S \in \mathcal{C}} d(S) \leq 2(|\mathcal{C}_a| + |\mathcal{C}_i| - 1)$.

Assume there is a leaf vertex $v \in V(H)$ connected with edge e . This vertex corresponds to the inactive set $C_v \in \mathcal{C}$. This set C_v is included in one of the connected components $N \in F$, where F is the set of vertices before pruning. The fact that $v \in V(H)$ is a leaf implies that all edges connecting C_v with other vertices from N but the edge e were excluded during pruning.

If the edge e is excluded from the component N , which is a tree itself, the component splits into N_1 and N_2 . Without loss of generality assume $C_v \subseteq N_1$. The edge e was not pruned, so $f(N_1) = 1$ or $f(N_2) = 1$. Due to $f(N) = 0$, the only possible variant is $f(N_1) = f(N_2) = 1$. Other cases are contradictory to Lemma 1.

Denote components $(C_v, C_1, \dots, C_m)$ derived from N_1 if edges from F' are used. For every component but C_v $f(C_i) = 0$ $i \neq v$ because the edges were pruned. But $f(C_v) = 0$ according to the original assumption. Hence $f(N_1) = f(\bigcup_{i=1}^m C_i \cup C_v) = 0$ by symmetry,

which is contradictory to $f(N_1) = 1$. So there are no leaf vertices in H which correspond to inactive vertices in the original graph, hence power of every inactive vertex is greater or equal to two or equal to zero. In the latter case the vertices do not affect the target inequality.